

Lab 6
Functions

Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw







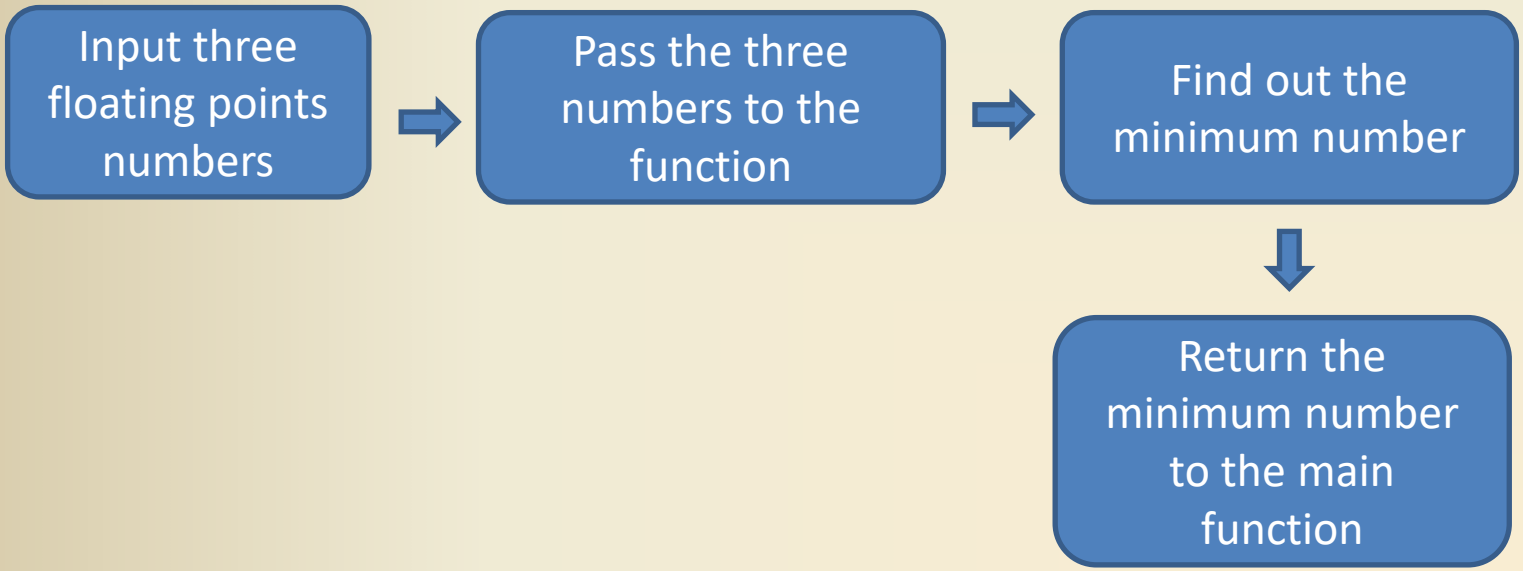
Lab 6.1
P113021 Minimum Number (30%)

- Test target: functions
- You are forced to use a function *double minimum(…)* to find the minimum of three floating-point numbers.
- You need to return the value from the function instead of using the global variables.



Lab 6.1
P113021 Minimum Number (30%)

- Flow chart:



Lab 6.1
P113021 Minimum Number (30%)

- function *double minimum(…)*:

```
double minimum(double x, double y, double z) {
    double min = x;

    if (y < min)
        min = y;

    if (z < min)
        min = z;

    return min;
}
```







Lab 6.2

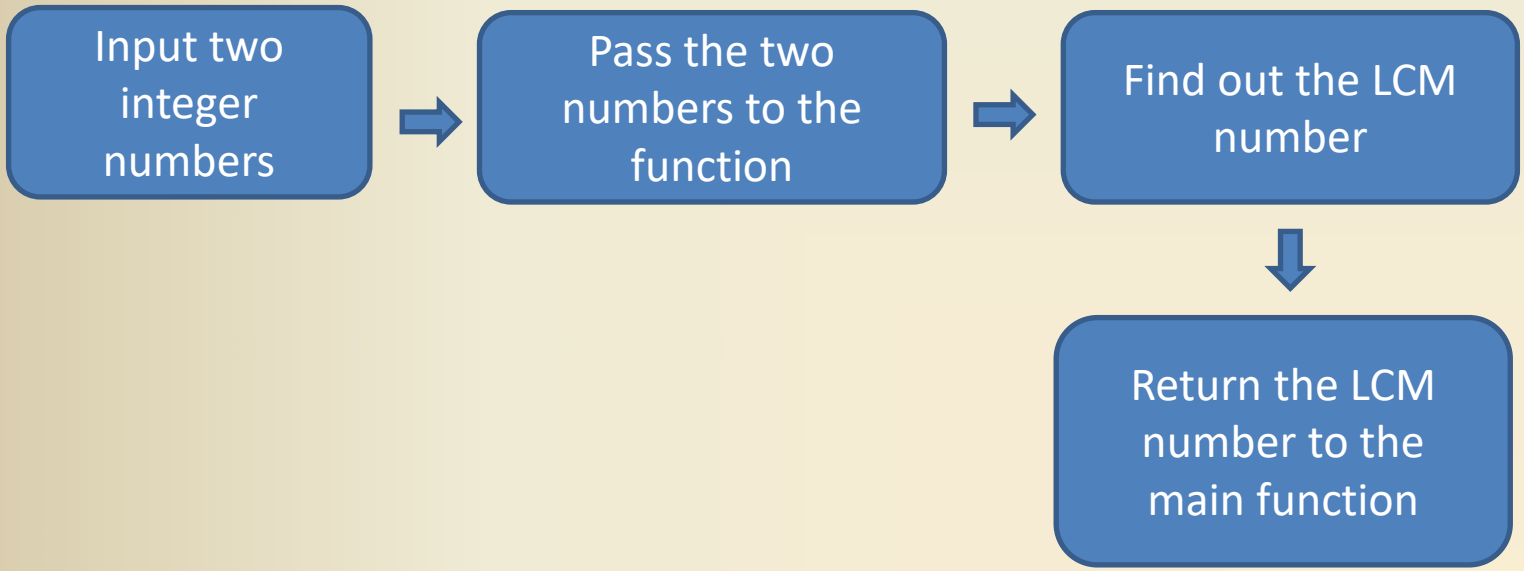
P113022 Least Common Multiple (LCM) (30%)

- Test target: functions
- You are forced to use a function *int LCM_calculator(…)* to find the LCM of two given numbers.
- You need to return the value from the function instead of using the global variables.



Lab 6.2
P113022 Least Common Multiple (LCM) (30%)

- Flow chart:



Lab 6.2
P113022 Least Common Multiple (LCM) (30%)

- How to find out the LCM number?
  - Determine the larger number
  - Start the while loop:
    - Creates an infinite loop that continues until the LCM is found.
    - The purpose of the loop is to incrementally check if the current value of max is divisible by both a and b. When this condition s met, the function returns the LCM.
    - Return LCM.

```
int LCM_calculator(int a, int b) {
    int max = 0;

    max = (a > b) ? a : b;

    while (true) {
        if (max % a == 0 && max % b == 0) {
            return max;
            break;
        } else
            ++max;
    }
}
```



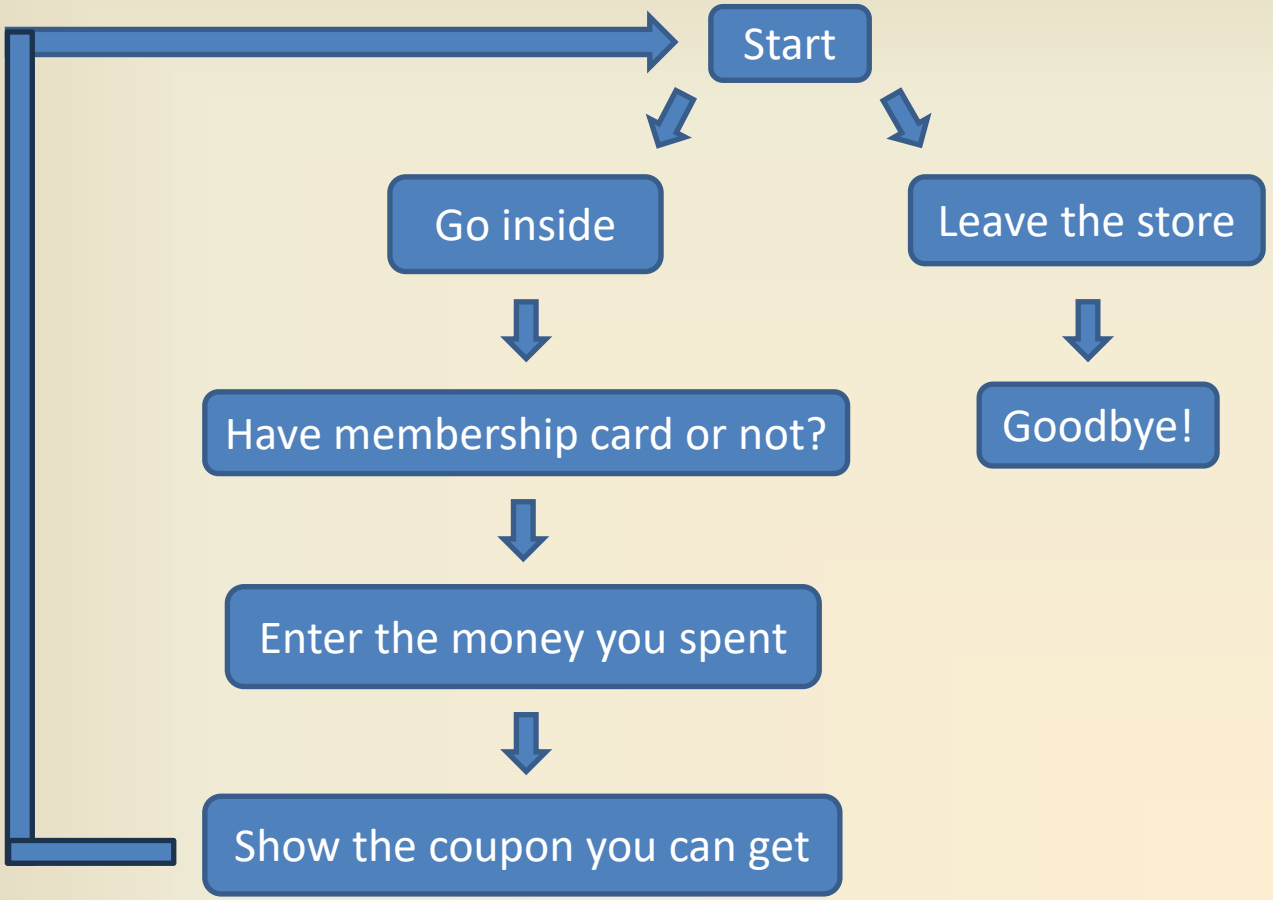
Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)

- Test target: functions
- In the NCU Electrical Appliance Store, the amount of coupons you receive depends on the money you spend. Additionally, having a membership affects the amount of coupons you get.
- You need to use at least 3 functions in your program.



Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)

- Flow chart:



Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)



- Three functions:
  - *bool enterStore();* // Function to handle entering the store
  - *bool hasMembershipCard();* // Function to check membership status
  - *void calculateRewards(bool hasMembership, int spending);* // Function to calculate and display rewards



Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)

- Main function:
  - Call the function *enterStore()* to determine whether to enter the store or leave.

```
// Ask user whether they want to enter or leave the store
if (!enterStore()) {
    cout << "********************** GOOD BYE! ************************" << endl;
    break; // Exit the program if user chooses to leave the store
}
```

  - Call the function *hasMembershipCard()* to ask for membership status.

```
// Ask for membership status
bool hasMembership = hasMembershipCard();
```

  - Call the function *calculateRewards()* to calculate the coupons you will receive based on membership status.

```
// Calculate rewards based on membership status and spending
calculateRewards(hasMembership, spending);
```



Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)

- Function *enterStore()* :

```
// Function to handle entering the store
bool enterStore() {
    string enterStoreChoice;
    cout << "*** NCU Electrical Appliance Store ***" << endl;
    cout << "Do you want to go inside or leave the store(Y/N): ";
    cin >> enterStoreChoice;

    if (enterStoreChoice == "N" || enterStoreChoice == "n") {
        return false; // Leave the store
    }

    return true; // Enter the store
}
```



Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)

- Function *hasMembershipCard()* :

```
// Function to check membership status
bool hasMembershipCard() {
    string membershipStatus;
    cout << "Do you have the membership card(Y/N): ";
    cin >> membershipStatus;

    if (membershipStatus == "Y" || membershipStatus == "y") {
        return true; // Customer has membership
    }

    return false; // Customer does not have membership
}
```



Lab 6.3
P113023 NCU Electrical Appliance Store (LCM) (30%)

- Function *calculateRewards()* :

```
// Function to calculate and display rewards
void calculateRewards(bool hasMembership, int spending) {
    if (hasMembership) {
        // Case when customer has membership
        if (spending < 5000) {
            cout << "***Congratulations !! You get 500 dollars coupon!!" << endl;
        } else if (spending >= 5000 && spending <= 10000) {
            cout << "***Congratulations !! You get 1000 dollars coupon!!" << endl;
        } else if (spending > 10000) {
            cout << "***Congratulations !! You get 3000 dollars coupon and one lottery ticket!!" << endl;
        }
    } else {
        // Case when customer does not have membership
        if (spending < 7000) {
            cout << "***Oh no!! You do not have any bonus to get…." << endl;
        } else if (spending >= 7000 && spending <= 10000) {
            cout << "***Congratulations !! You get 500 dollars coupon!!" << endl;
        } else if (spending > 10000) {
            cout << "***Congratulations !! You get 1500 dollars coupon and become a store member!!" << endl;
        }
    }
}
```

Lab 6.4
P113024 The People's Commune (20%)

- Test target: functions
- Assume you have only two pots to measure the amount of rice. The two pots have capacities of X liters and Y liters (where X and Y are integers and $X \neq Y$). To measure exactly Z liters of rice, where Z is a positive integer and $Z \leq X + Y$, the kitchen staff can only do the following:
  - Fill any pot completely.
  - Empty any pot completely.
  - Pour rice from one pot into the other until the first pot is empty or the other pot is full.
- You need to determine whether it is possible to measure exactly Z liters of rice.



Lab 6.4
P113024 The People's Commune (20%)

- Step 1: Check if Z is greater than the total capacity of both pots.
- Step 2: Determines if Z is a multiple of the greatest common divisor (GCD) of the two pot capacities.

```
bool canMeasureWater(int X, int Y, int Z) {

    if (Z > X + Y)
        return false;

    int g = gcd(X, Y);

    if (Z % g == 0)
        return 1;
    else
        return 0;
}
```

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```



Lab 6.4
P113024 The People's Commune (20%)

- Why GCD?
  - The operations you can perform essentially allow you to create any volume that is a linear combination of *X* and *Y.* Mathematically, this means:

    $$Z = mX + nY$$

    Where *m* and *n* are integers (which can be positive, negative, or zero).
  - Bézout's identity :
    - Let X and Y be **integers** with **greatest common divisor** Z. Then there exist integers m and n such that mX + nY = Z. Moreover, the integers of the form uX + vY are exactly the multiples of Z.
  - By calculating the GCD of *X* and *Y*, you can determine whether *Z* is a multiple of this GCD. If it is, then *Z* liters can be measured using the two pots; otherwise, it cannot.







Lab 6.4
P113024 The People's Commune (20%)

- Reference:
  https://en.wikipedia.org/wiki/B%C3%A9zout%27s_identity







Evaluation

- Lab6.1(30%)
  - 5 testcases, 6% each
- Lab6.2(30%)
  - 5 testcases, 6% each
- Lab6.3(30%)
  - 5 testcases, 6% each
- Lab6.4(20%)
  - 5 testcases, 4% each







Andy, Yu-Guang Chen

Assistant Professor, Department of EE, NCU

Email: andyygchen@ee.ncu.edu.tw